RAG Interview Questions and Answers

Simple answers you can say in your own words
Generative AI / AI Engineer interviews

How to use this: Read each question, answer out loud first, then compare with the answer here. The 'Say in interview' box is a short version you can speak in 20 to 30 seconds. Questions are based on common topics in AI engineer interviews. They are practice questions, not leaked questions from any company.

Part 1: Basics

Q1. What is RAG?

Answer:

RAG means Retrieval Augmented Generation. The LLM does not know our private data or new data. So when the user asks a question, we first search our own documents, take the useful parts, and give them to the LLM along with the question. The LLM then writes the answer using that data.

Say in interview: RAG means we first retrieve the related data from our documents, then give it to the LLM, so the LLM answers from our real data.

Q2. Why do we use RAG instead of only using an LLM?

Answer:

Because an LLM alone has some problems:

- It does not know our private data (company documents, policies).
- Its knowledge is old. It was trained up to a past date.
- It can make up wrong answers. This is called hallucination.
- Training the model again every time data changes is very costly.

Say in interview: RAG gives real and latest data to the LLM, reduces wrong answers, and is much cheaper than training again.

Q3. Explain how RAG works step by step.

Answer:

There are two phases. First we prepare the data. Then we answer the questions.

- **Preparing data (one time):** take documents, cut them into small chunks, convert each chunk into an embedding (numbers), store in a vector database.
- **Answering (every question):** convert the question into an embedding, search the vector database for the most similar chunks, put those chunks and the question in the prompt, send to the LLM, and the LLM gives the answer.

Say in interview: I cut the documents into chunks, make embeddings, and store them in a vector DB. When a question comes, I find the closest chunks, send them to the LLM with the question, and the LLM writes the answer.

Q4. What is an embedding?

Answer:

An embedding is a list of numbers that represents the meaning of a text. Texts with similar meaning have similar numbers. For example, 'refund policy' and 'return money' will be close to each other, even though the words are different. A common size is 1536 numbers (dimensions).

Say in interview: An embedding converts text into numbers so that similar meaning gives similar numbers, and the computer can search by meaning.

Q5. What is a vector database? How is it different from a normal database?

Answer:

A vector database stores embeddings and can quickly find the closest ones to a given embedding. Examples: Pinecone, Chroma, FAISS, Weaviate, Milvus, and pgvector (inside Postgres).

- Normal database: searches by exact match, like ID = 101 or name = 'John'.
- Vector database: searches by meaning or similarity, like 'find texts similar to this question'.

Say in interview: A normal DB finds exact matches. A vector DB finds the most similar meaning using the embeddings.

Q6. What is chunking? Why is it needed?

Answer:

Chunking means cutting a big document into small pieces before making embeddings. It is needed because the LLM has a limit on input size, and because small focused pieces give better search results than a whole big document.

- Chunk too big: meaning gets mixed up, search is less accurate, and it uses more tokens.
- Chunk too small: it loses context and the answer may be incomplete.
- A common starting point is 300 to 500 tokens. Then we test and adjust.

Say in interview: I cut documents into small chunks, usually a few hundred tokens, and I test different sizes to see what gives the best answers.

Q7. What is chunk overlap?

Answer:

Overlap means the end of one chunk is repeated at the start of the next chunk, for example 10 to 20 percent. This is done so that a sentence or idea is not cut in the middle and the meaning is not lost.

Say in interview: Overlap keeps a small part common between two chunks so that no important meaning is lost at the cut.

Q8. What is cosine similarity?

Answer:

It is a way to measure how close two embeddings are. It checks the angle between the two vectors. If the angle is small, the meaning is similar. The score is near 1 for very similar and near 0 for not related.

Say in interview: Cosine similarity tells how similar two embeddings are. Higher score means more similar meaning.

Q9. What is top-k?

Answer:

Top-k means how many best matching chunks we take from the vector database. For example, k = 5 means we take the 5 most similar chunks and send them to the LLM.

- k too small: we may miss the right chunk.
- k too big: we add noise, cost goes up, and the LLM can get confused.

Say in interview: Top-k is the number of best chunks I retrieve. I set it by testing, and often I retrieve more and then re-rank to keep the best few.

Part 2: Improving Quality

Q10. What is hybrid search?

Answer:

Hybrid search means using two searches together: vector search (by meaning) and keyword search like BM25 (by exact words). Vector search is good for meaning but can miss exact words like product codes, names, or error numbers. Keyword search catches those. We combine both results.

Say in interview: Hybrid search combines meaning search and keyword search, so we get both related meaning and exact words.

Q11. What is a re-ranker?

Answer:

First, the vector search quickly gets many chunks, for example 20. Then a re-ranker model looks at the question and each chunk together more carefully and gives a better score. We keep only the top few, for example 5. It is slower but more accurate, so we use it only on the small set.

Say in interview: I retrieve more chunks first, then a re-ranker checks them carefully and I keep only the best few. This improves accuracy.

Q12. What is query rewriting?

Answer:

Users often write short or unclear questions, like 'what about the refund?'. Before searching, we can use the LLM to rewrite the question into a clear one, or into a few different versions. This helps to find better chunks.

Say in interview: I rewrite unclear user questions into clear ones before searching, so the retrieval gets better.

Q13. What is metadata filtering?

Answer:

Along with each chunk, we store extra information like document name, date, department, or user access. When searching, we can filter by this. For example, search only in 'HR documents' or only documents from this year. It makes the search faster and more correct.

Say in interview: I store metadata with each chunk and filter by it, like department or date, so the search is limited to the right data.

Q14. What is the 'lost in the middle' problem?

Answer:

When we send many chunks to the LLM, it pays more attention to the chunks at the start and the end, and it can miss the ones in the middle. To reduce this, send fewer and better chunks, use a re-ranker, and put the most important chunks at the start or end.

Say in interview: The LLM can ignore the chunks in the middle of a long context, so I send fewer, better chunks and put the best ones first.

Q15. How do you reduce hallucination in RAG?

Answer:

Hallucination means the LLM gives a wrong answer with confidence.

- Retrieve good chunks (fix chunking, hybrid search, re-ranker).
- Tell the LLM in the prompt: 'Answer only from the given context. If the answer is not there, say I don't know.'
- Show the source document with the answer so the user can verify.
- Keep temperature low for factual answers.
- Check the answer against the retrieved chunks.

Say in interview: I give good context, tell the LLM to answer only from it and say 'I don't know' otherwise, and show the sources.

Part 3: Debugging and Evaluation

Q16. Users say the RAG answers are wrong. How do you debug?

Answer:

I check the retrieval first. If the right chunk is not coming, the LLM cannot give the right answer.

- **Retrieval problems:** chunk size is wrong, a different embedding model is used for documents and questions (both must use the same model), exact words are missed (use hybrid search), top-k is too low (add a re-ranker).
- **Generation problems:** the prompt does not say to answer only from context, or too much context is sent.
- I also make a test set of questions with known answers to find where it fails.

Say in interview: First I check if the right chunk was retrieved. If not, I fix chunking, embeddings, or search. If yes but the answer is wrong, I fix the prompt and the context.

Q17. How do you evaluate a RAG system?

Answer:

I check two parts separately: retrieval and generation.

- **Retrieval:** is the correct chunk in the top-k results? (metrics like recall and precision).
- **Generation:** is the answer correct, and does it come only from the retrieved context? This is called faithfulness. Is it answering the actual question? This is relevance.
- Tools like RAGAS can help. Also keep a test set of questions and answers, and test again whenever we change something.

- Collect real user feedback (thumbs up and down).

Say in interview: I test retrieval and answer quality separately, using a test set of questions with known answers, and I also collect user feedback.

Q18. Same question gives different quality answers sometimes. Why?

Answer:

There can be many reasons: the LLM temperature is high, retrieval returns different chunks for slightly different wording of the question, or the data in the DB changed. To fix: lower the temperature, use query rewriting, and keep a fixed test set to compare.

Say in interview: It can be due to high temperature or different retrieved chunks. I lower the temperature and improve retrieval.

Part 4: Real World and Production

Q19. How do you handle PDFs that have tables or images?

Answer:

Normal text extraction can break tables and miss images. Better ways:

- Use a good PDF parser or OCR that keeps the table structure.
- Keep a table as one chunk, do not cut it in the middle.
- For images or charts, use a vision model to write a text description, and store that text.
- Store the page number in metadata so we can show the source.

Say in interview: I use a good parser, keep tables together as one chunk, and convert images into text descriptions using a vision model.

Q20. How do you keep the data updated in RAG?

Answer:

When a document changes, we only re-process that document: delete its old chunks and embeddings and add the new ones. We do not rebuild everything. We can run this on a schedule or when a change happens. This is a big benefit of RAG over fine-tuning.

Say in interview: I update only the changed documents in the vector DB, so there is no need to retrain the model.

Q21. How do you secure a RAG system? What about data access?

Answer:

Not every user should see every document. So:

- Use metadata filters or access control so a user only retrieves chunks they are allowed to see. Do the check before sending the chunks to the LLM.
- **Prompt injection:** a document or user message may contain hidden instructions like 'ignore the previous rules'. Treat retrieved text as data, not as instructions. Validate the outputs and limit what the system can do.
- Do not send sensitive data like passwords or personal details to the LLM unless needed. Keep logs.

Say in interview: I filter the data by user permission before retrieval, and I treat retrieved text as data and not as instructions, to avoid prompt injection.

Q22. How do you reduce the cost and delay (latency) in RAG?

Answer:

- Send fewer and better chunks, so fewer tokens are used.
- Cache the answers for common questions.
- Use a smaller and faster model for simple questions.
- Use streaming so the user sees the answer start quickly.
- Use a re-ranker only on a small set of chunks.

Say in interview: I reduce tokens by sending fewer good chunks, cache common questions, and use smaller models where possible.

Part 5: Comparison Questions

Q23. What is the difference between RAG and fine-tuning?

Answer:

The difference is where the new information goes. In RAG, the model does not change. We give the data in the prompt when the user asks. It is like an open book exam. In fine-tuning, we train the model and change its weights. It is like studying before the exam.

- **Use RAG when:** data changes often, data is private, or you want to show the source.
- **Use fine-tuning when:** you want a fixed style or format, or a small model for one specific task.
- Many real projects use both together.

Say in interview: For facts and knowledge I start with RAG. I use fine-tuning for style and behavior. Fine-tuning is costly and facts become old.

Q24. What is agentic RAG?

Answer:

In normal RAG, we search one time and then answer. In agentic RAG, an AI agent decides by itself: does it need to search at all? What to search? Is the result good enough, or should it search again with a different query? It can also use many sources and tools. It is more powerful for hard questions, but it is slower and costs more.

Say in interview: Agentic RAG lets the agent decide when and what to search, and search again if the result is not good enough.

Q25. When would you NOT use RAG?

Answer:

When the data is very small, we can put it directly in the prompt. When the task is only about style or format, fine-tuning or a good prompt is enough. When the answer needs exact numbers from a database, a direct SQL query or a tool is better than searching text chunks.

Say in interview: If the data is small, or I need exact database values, I would use the prompt or a tool instead of RAG.

If you do not know an answer, say honestly: 'I have not worked on this directly, but my understanding is...' and then explain what you know. It is better than guessing.